Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

Hardware-Software-Systeme

Kapitel 04 - VHDL - Entwurf digitaler Schaltungen

Prof. Dr.-Ing. Guillermo Payá Vayá, 14. Dezember 2025

Speicherkomponenten

Speicherkonzept: RAM vs. Register Banks

- Konzeptionell ist eine Registerbank nicht dasselbe wie ein RAM (Random Access Memory), da eine Registerbank eine zweidimensionale Anordnung von Flipflops und ein RAM eine zweidimensionale Anordnung von Speicherzellen ist.
  $\rightarrow$ Dies ist jedoch ein Implementierungsunterschied und auf der Abstraktionsebene, die für den HDL-Entwurf verwendet wird, gibt es keinen Unterschied.
- Ein RAM oder ein Register kann durch die Verwendung des folgenden Signaltyps abgeleitet werden:

```
type memory_type_t is array (0 to 2**address_size_g-1) of
  std_ulogic_vector(word_size_g-1 downto 0);
signal memory : memory_type_t;
```

Asynchronous Read / Synchronous Write

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity RAM is
  generic (word_size_g    : natural;
           address_size_g : natural);
  port (clk, write_i      : in  std_ulogic;
        adress_i          : in  std_ulogic_vector(adress_size_g-1 downto 0);
        D_i               : in  std_ulogic_vector(word_size_g-1 downto 0);
        Q_o               : out std_ulogic_vector(word_size_g-1 downto 0));
end entity RAM;
architecture RTL of RAM is
  type memory_type_t is array (0 to 2**address_size_g-1) of
    std_ulogic_vector(word_size_g-1 downto 0);
  signal memory : memory_type_t;
begin
  sync_write : process (clk)   -- <- it is not possible to use a reset signal!!
  begin
    if clk = '1' and clk'event then
      if write_i = '1' then
        memory(to_integer(unsigned(address_i))) <= D_i;
      end if;
    end if;
  end process sync_write;
  Q_o <= memory(to_integer(unsigned(address_i)));
end architecture RTL;
```

Synchronous Read / Synchronous Write / WaR

WaR -> Write after Read

```
entity RAM is
  generic (word_size_g    : natural;
           address_size_g : natural);
  port (clk, write_i      : in  std_ulogic;
        address_i         : in  std_ulogic_vector(address_size_g-1 downto 0);
        D_i               : in  std_ulogic_vector(word_size_g-1 downto 0);
        Q_o               : out std_ulogic_vector(word_size_g-1 downto 0));
end entity RAM;
architecture RTL of RAM is
  type memory_type_t is array (0 to 2**address_size_g-1) of
    std_ulogic_vector(word_size_g-1 downto 0);
  signal memory  : memory_type_t;
  signal Q_o_reg : std_ulogic_vector(word_size_g-1 downto 0);
begin
  sync_write : process (clk)   -- <- it is not possible to use a reset signal!!
  begin
    if clk = '1' and clk'event then
      if write_i = '1' then
        memory(to_integer(unsigned(address_i))) <= D_i;
      end if;
      Q_o_reg <= memory(to_integer(unsigned(address_i)));
    end if;
  end process sync_write;
  Q_o <= Q_o_reg;
end architecture RTL;
```

Synchronous Read / Synchronous Write / RaW

RaW -> Read after Write

```
entity RAM is
  generic (word_size_g    : natural;
           address_size_g : natural);
  port (clk, write_i      : in  std_ulogic;
        address_i         : in  std_ulogic_vector(address_size_g-1 downto 0);
        D_i               : in  std_ulogic_vector(word_size_g-1 downto 0);
        Q_o               : out std_ulogic_vector(word_size_g-1 downto 0));
end entity RAM;
architecture RTL of RAM is
  type memory_type_t is array (0 to 2**address_size_g-1) of
    std_ulogic_vector(word_size_g-1 downto 0);
    signal memory : memory_type_t;
    signal read_address : unsigned(address_i'range);
begin
  sync_write : process (clk)   -- <- it is not possible to use a reset signal!!
  begin
    if clk = '1' and clk'event then
      if write_i = '1' then
        memory(to_integer(unsigned(address_i))) <= D_i;
      end if;
      read_address <= unsigned(address_i);
    end if;
  end process sync_write;
  Q_o <= memory(to_integer(read_address));
end architecture RTL;
```

Reset

- In Speichermodellen, für die ein RAM erforderlich ist, können weder asynchrone noch synchrone Resets verwendet werden, da RAMs im Allgemeinen nicht über diese Funktionalität verfügen. Daher führen nur die nicht reset-baren Registervorlagen dazu, dass eine RAM-Inferenz stattfindet.
- Um einen RAM-Speicher zurückzusetzen, muss der Reset auf einer höheren Ebene im Entwurf durch Schreibvorgänge des Reset-Wertes an alle Adressen durchgeführt werden. Aber dies könnte dazu führen, dass kein RAM, sondern eine Registerbank synthetisiert wird.
- Eine Registerbank hingegen kann auf beide Arten zurückgesetzt werden, da sie nur ein adressierbares Feld gewöhnlicher Register ist.

Dual-Port Synchronous Read / Synchronous Write

```
entity RAM is
  generic (word_size_g    : natural;
           address_size_g : natural);
  port (clk, wr_a_i, wr_b_i       : in  std_ulogic;
        address_a_i, address_b_i : in  std_ulogic_vector(address_size_g-1 downto 0);
        DA_i, DB_i                : in  std_ulogic_vector(word_size_g-1 downto 0);
        QA_o, QB_o                : out std_ulogic_vector(word_size_g-1 downto 0));
end entity RAM;
architecture RTL of RAM is
  type memory_type_t is array (0 to 2**address_size_g-1) of
    std_ulogic_vector(word_size_g-1 downto 0);
    signal memory : memory_type_t;
    signal QA_o_reg, QB_o_reg : std_ulogic_vector(word_size_g-1 downto 0);
begin
  sync_write : process (clk)   -- <- it is not possible to use a reset signal!!
  begin
    if clk = '1' and clk'event then
      if wr_a_i = '1' then memory(to_integer(unsigned(address_a_i))) <= DA_i;
      end if;
      if wr_b_i = '1' then memory(to_integer(unsigned(address_b_i))) <= DB_i;
      end if;
      QA_o_reg <= memory(to_integer(unsigned(address_a_i));
      QB_o_reg <= memory(to_integer(unsigned(address_b_i));
    end if;
  end process sync_write;
  QA_o <= QA_o_reg;
  QB_o <= QB_o_reg;
end architecture RTL;
```

ROM vs. Decoder

- Ein ROM (Read Only Memory) ist eine weitere Struktur, die vom Synthese-Tool abgeleitet wird, so dass ein spezielles Template verwendet muss, um eine ROM-Instanz zu erhalten.
- Wenn die Zieltechnologie keine ROMs unterstützt oder das Synthese-Tool keinen ROM implementieren kann, das den Designanforderungen entspricht, wird es stattdessen als Dekodierlogik implementiert.
- Der Unterschied zwischen ROMs und RAMs besteht darin, dass man kein allgemeines ROM entwerfen und dann überall wiederverwenden kann, wo man es braucht, und zwar aus dem einfachen Grund, dass jedes ROM einen anderen Dateninhalt hat und der Inhalt im Entwurf fest einkodiert ist. Daher muss für jede ROM im Entwurf eine andere Entität und Architektur unter Verwendung derselben ROM-Vorlage entworfen werden.

(1) Case-Anwesung Decoder

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity ROM is
  generic (word_size_g    : natural := 3;
           address_size_g : natural := 3);
  port (address_i : in  std_ulogic_vector(address_size_g-1 downto 0);
        Q_o       : out std_ulogic_vector(word_size_g-1 downto 0));
end;
architecture RTL of ROM is
begin
  read_process: process(address_i)
  begin
    case address_i is
      when "000" =>  Q_o <= "000";
      when "001" =>  Q_o <= "001";
      when "011" =>  Q_o <= "010";
      when "010" =>  Q_o <= "011";
      when "110" =>  Q_o <= "100";
      when "111" =>  Q_o <= "101";
      when "101" =>  Q_o <= "110";
      when "100" =>  Q_o <= "111";
      when others => Q_o <= "XXX";
    end case;
  end process read_process;
end architecture RTL;
```

(2) Table Lookup Decoder

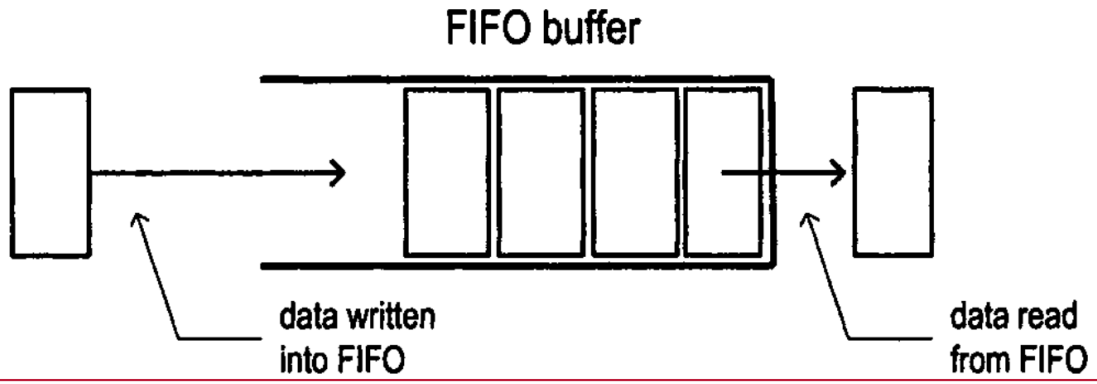
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity ROM is
  generic (word_size_g    : natural := 3;
           address_size_g : natural := 3);
  port (address_i : in  std_ulogic_vector(address_size_g-1 downto 0);
        Q_o       : out std_ulogic_vector(word_size_g-1 downto 0));
end;
architecture RTL of ROM is
  type memory_type_t is array (0 to 7) of std_ulogic_vector(2 downto 0);
  constant memory : memory_type_t :=
    ("000", "001", "011", "010", "110", "111", "101", "100");
begin

  Q_o <= memory(to_integer(unsigned(address_i)));

end architecture RTL;
```

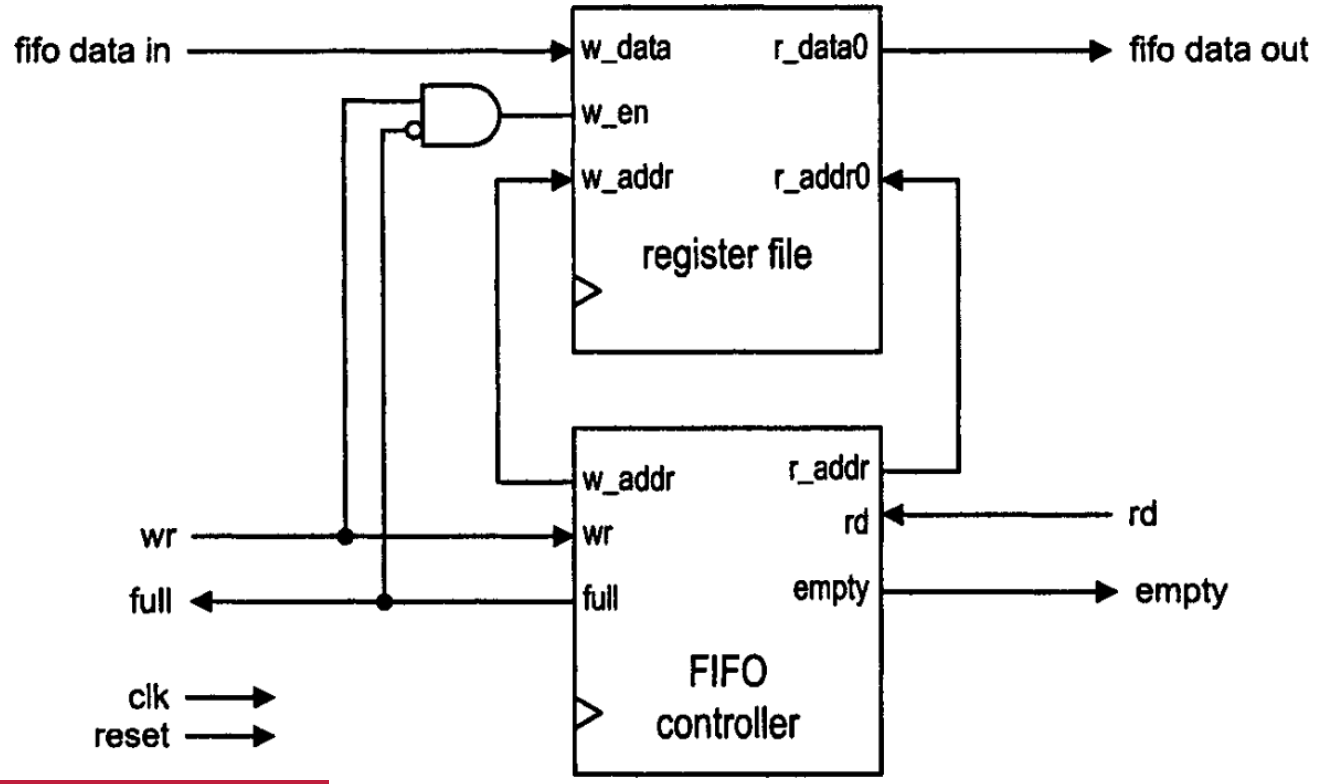
First-in-first-out Memory (FIFO)

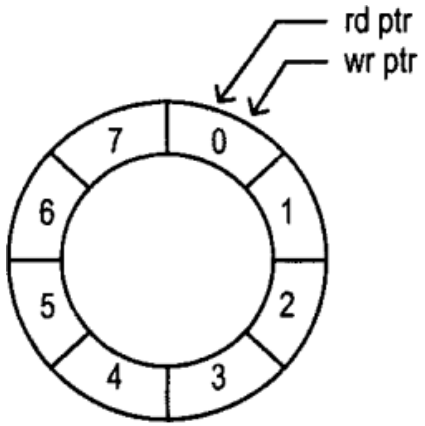
- Ein *first-in-first-out* (FIFO)-Buffer fungiert als ëlastischerSSpeicher zwischen zwei Teilsystemen.
- Ein Teilsystem speichert (d. h. schreibt) Daten in den FIFO, und das andere Teilsystem ruft Daten aus der FIFO ab (d. h. liest) und entfernt sie aus der FIFO
- Die Reihenfolge des Datenabrufs entspricht der Reihenfolge der gespeicherten Daten, daher wird der Buffer als *First-in-First-out* bezeichnet.
- Ein anderer alternativer Buffer ist der *last-in-first-out* (LIFO).



FIFO Kontrolllogik

- Die häufigste Art, einen FIFO-Buffer zu bauen, ist das Hinzufügen einer einfachen Schaltlogik zu einem generischen Speicherarray, z. B. einem Register oder einem RAM.





FIFO-Steuerungsschaltung mit zwei Zählern (I)

```
library ieee ;
use ieee.std-logic-1164.all;
use ieee.numeric-std.all;

entity fifo-sync-counter is
  port(clk, reset:    in  std_ulogic;
       wr, rd:        in  std_ulogic;
       full, empty :  out std_ulogic;
       w_addr, r_addr: out std_ulogic_vector (1 downto 0));
end entity fifo_sync_counter;
architecture RTL of fifo-sync-counter is
  constant N: natural := 3;
  signal w_ptr_reg, w_ptr_next: unsigned(N-1 downto 0);
  signal r_ptr_reg, r_ptr_next: unsigned(N-1 downto 0);
  signal full_flag, empty_flag: std_ulogic;
begin
  -- register
  sync: process (clk, reset)
  begin
    if (reset = '1') then
      w_ptr_reg <= (others => '0');
      r_ptr_reg <= (others => '0');
    elsif (clk'event and clk='1') then
      w_ptr_reg <= w_ptr_next;
      r_ptr_reg <= r_ptr_next;
    end if;
  end process sync;
```

FIFO-Steuerungsschaltung mit zwei Zählern (II)

```
  -- write pointer next-state logic
  w_ptr_next <= w_ptr_reg+1 when (wr='1' and full_flag='0') else
                w_ptr_reg;
  full_flag  <= '1' when w_ptr_reg = r_ptr_reg-1 else '0';
  -- write port outputs
  w_addr <= std_ulogic_vector(w_ptr_reg);
  full <= full_flag;

  -- read pointer next-state logic
  r_ptr_next <= r_ptr_reg+1 when (rd='1' and empty_flag='0') else
                r_ptr_reg;
  empty_flag <= '1' when r_ptr_reg = w_ptr_reg else '0';
  -- read port outputs
  r_addr <= std_ulogic_vector (r_ptr_reg);
  empty <= empty_flag;
end architecture RTL;
```

Tri-State-Modellierung

Datentypen `std_ulogic` und `std_logic`

Wert	Bedeutung	Verwendung
'U'	Nicht initialisiert	Nicht initialisiertes Signal im Simulator
'X'	Undefiniert	Simulator erkennt mehr als einen aktiven Signaltreiber (Buskonflikt)
'0'	Starke logische '0'	entspricht '0' eines bit-Signals
'1'	Starke logische '1'	entspricht '1' eines bit-Signals
'Z'	Hochohmig 'Z'	Tri-State-Ausgang
'W'	Schwach unbekannt	Simulator erkennt Buskonflikt zwischen 'L' und 'H'
'L'	Schwache logische '0'	Ausgang mit Pull-Down-Widerstand (Open Source)
'H'	Schwache logische '1'	Ausgang mit Pull-Up-Widerstand (Open Drain)
'-'	Don’t-Care	Logikzustand bedeutungslos; kann für Minimierung verwendet werden

Datentyp `std_ulogic`

- Für Signale des Typs `std_ulogic` dürfen jeweils nur ein Treiber existieren, d.h. diese Signale dürfen innerhalb einer `architecture` nur in einem Prozess, einer nebenläufigen Anweisung oder einer Komponenteninstanz eine Wertzuweisung erfahren, auch wenn die Signalzuweisung im VHDL-Code zu unterschiedlichen Zeitpunkten erfolgt.
- Zu berücksichtigen ist nämlich, dass alle Treiber zu jedem Zeitpunkt einen Signalwert liefern. Signalkonflikte, die in der Hardware zu Schaltungsfehlern führen würden, werden bei diesem Datentyp also nicht aufgelöst (unresolved data type, daher das **u** in der Typbezeichnung).
- Allerdings erkennt der Compiler diese Situation und bricht mit einer Fehlermeldung ab.

Datentyp `std_logic`

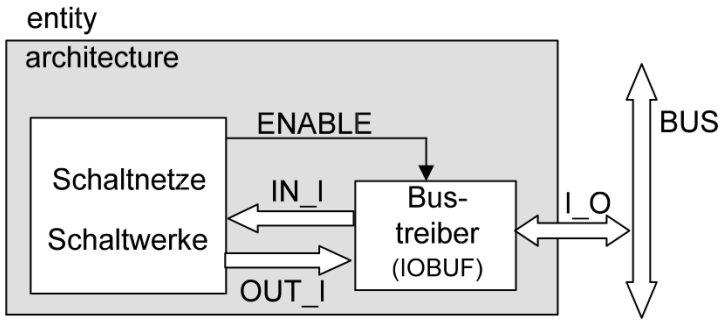
- Beim Datentyp `std_logic` dürfen durchaus mehrere Treiber für ein Signal existieren.
- Anwendung findet dieser Datentyp insbesondere bei bidirektionalen Bussen, auf denen die Daten von mehreren Sendern stammen können. Die Entscheidung, welcher Treiber sich durchsetzt, trifft der Simulator mit der im 1164-IEEE-Standard definierte Auflösungsfunktion (Resolution Function).
- Für den Anwender ist nur wichtig, zu wissen, wie die Auflösungsfunktion des Datentyps `std_logic` wirkt.

Datentypen `std_logic`: Auflösungsfunktion

	'U'	'X'	'0'	'1'	'Z'	'W'	'L'	'H'	'-'
'U'	'U'	'U'	'U'	'U'	'U'	'U'	'U'	'U'	'U'
'X'	'U'	'X'	'X'	'X'	'X'	'X'	'X'	'X'	'X'
'0'	'U'	'X'	'0'	'X'	'0'	'0'	'0'	'0'	'X'
'1'	'U'	'X'	'X'	'1'	'1'	'1'	'1'	'1'	'X'
'Z'	'U'	'X'	'0'	'1'	'Z'	'W'	'L'	'H'	'X'
'W'	'U'	'X'	'0'	'1'	'W'	'W'	'W'	'W'	'X'
'L'	'U'	'X'	'0'	'1'	'L'	'W'	'L'	'W'	'X'
'H'	'U'	'X'	'0'	'1'	'H'	'W'	'W'	'H'	'X'
'-'	'U'	'X'	'X'	'X'	'X'	'X'	'X'	'X'	'X'

Tri-State-Ausgangsstufen

```
library IEEE;
use ieee.std_logic_1164.all;
entity BUSTREIBER is
  port(enable_I : in  std_ulogic;
       I_O    : inout std_logic_vector(1 downto 0);
       OUT_I  : in    std_ulogic_vector(1 downto 0);
       IN_I   : out   std_ulogic_vector(1 downto 0));
end entity BUSTREIBER;
architecture TRI_STATE_OUT of BUSTREIBER is
begin
  -- Gesteuerter Schreibvorgang
  WRITE: process (enable, OUT_I)
  begin
    if enable_I = '1' then
      I_O <= To_StdLogicVector(OUT_I);
    else
      I_O <= (others => 'Z');
    end if;
  end process WRITE;
  -- Paralleler Lesevorgang
  READ: IN_I <= To_StdULogicVector(I_O);
end architecture TRI_STATE_OUT;
```



Literatur

- Jürgen Reichardt, Bernd Schwarz, "VHDL-Simulation und -Synthese", Walter de Gruyter GmbH, 2020
- Peter J. Ashenden, "The Designer's Guide to VHDL", Morgan Kaufmann, 2010.
- Janick Bergeron, "Writing Testbenches: Functional Verification of HDL Models", Springer Science & Business Media, 2012.